

Подготовка к экзамену по C++ (ИТМО)

Тема 2: Структуры, объединения и выравнивание ПАМЯТИ

Hardcore Theory Edition

14 июня 2026 г.

1 Структуры (struct) и классы (class)

В языке Си структуры были просто мешками для данных. В C++ структуры **ничем не отличаются** от классов, кроме модификатора доступа по умолчанию.

- В `class` по умолчанию всё **private** (закрыто).
- В `struct` по умолчанию всё **public** (открыто).
- И то, и другое может иметь конструкторы, деструкторы, методы и виртуальные функции.

Вопрос на 5: Какой размер имеет пустая структура?

`struct Empty {};` занимает **1 байт**. В C++ каждый объект должен иметь уникальный адрес в памяти. Если бы пустая структура весила 0 байт, то массив из 10 пустых структур указывал бы на один и тот же адрес, что ломает логику указателей. Исключение — EBO (Empty Base Optimization), о котором мы говорили в умных указателях.

2 Выравнивание памяти (Padding / Alignment)

Это самая важная подтема. Процессор не читает память по одному байту. На 64-битной архитектуре он читает память "машинными словами" по 8 байт за раз. Если переменная типа `int` (4 байта) пересекает границу такого слова (половина в одном машинном слове, половина в другом), процессору придется сделать два чтения и склеить результат. Это очень медленно.

Поэтому компилятор вставляет **невидимые "пустые" байты (padding)** между полями структуры, чтобы каждая переменная лежала по "удобному" адресу.

Правило выравнивания: Адрес переменной должен быть кратен её размеру.

- `char` (1 байт) может лежать где угодно.
- `int` (4 байта) должен лежать по адресу, кратному 4 (0, 4, 8, 12...).
- `double` (8 байт) должен лежать по адресу, кратному 8 (0, 8, 16...).
- Размер **всей структуры** округляется так, чтобы быть кратным размеру **самого большого её поля**.

Суть на пальцах (Жизненная аналогия)

Представьте, что память компьютера — это полки на складе (каждая полка вмещает 8 коробок). `char` — это спичечный коробок, `double` — это огромный сейф на 8 ячеек. Если вы положили спичечный коробок в начало полки, сейф туда уже не влезет (он должен стоять на одной полке целиком, чтобы грузчик забрал его за один раз). Поэтому грузчик (компилятор) оставит 7 ячеек на первой полке ПУСТЫМИ (padding) и поставит сейф на новую, вторую полку. Из-за этого вы тратите склад впустую.

Как оптимизировать: Располагайте поля в структуре **по убыванию их размера** (сначала 8 байт, потом 4, потом 2, потом 1). Тогда пустых дыр не будет.

3 Объединения (union)

`union` — это структура, в которой **все поля делят один и тот же кусок памяти**.

- Размер объединения равен размеру его **самого большого** поля.
- Вы можете записывать данные только в одно поле за раз. Если вы записали `int`, а потом прочитали из этого же `union` поле `float`, компилятор просто возьмет биты `int` и интерпретирует их как `float`. В C++ чтение неактивного поля `union` официально считается **Undefined Behavior** (хотя компиляторы GCC и Clang разрешают это для Type Punning).

Суть на пальцах (Жизненная аналогия)

`union` — это гостиничный номер. В нём может жить либо бизнесмен (`int`), либо рок-звезда (`double`), но **не одновременно**. Кровать (память) у них одна. Если бизнесмен уехал, а рок-звезда заехала, старые вещи бизнесмена, оставшиеся в номере, будут использоваться рок-звездой не по назначению.

4 Битовые поля (Bit-fields)

Иногда нам нужен даже не байт, а всего 1 бит (например, для флагов). C++ позволяет указать точное количество бит для поля с помощью двоеточия: `unsigned int is_active : 1;`

- Несколько битовых полей подряд "упаковываются" компилятором в одну переменную (например, в один `char` или `int`).
- Взять указатель на битовое поле **нельзя** (в C++ нет указателей на отдельные биты).